AMAZON MERCHANT CRM

v2.0 — Enterprise Multi-Project Sales & Onboarding CRM

Admin → Manager → Team hierarchy · Real-time live dashboards · Dynamic form builder ·
Bulk lead upload · PostgreSQL + Redis + Celery + Django Channels · Cursor AI build spec

Prepared for: Cursor AI Development | Generated: July 11, 2026

1. Vision & What's New in v2.0

This is the upgraded specification for the Amazon Merchant CRM — evolving it from a single-tier BDM/TL/Manager tool into a full **Company → Admin → Manager → Team** sales operations platform, positioned to be simpler and faster than Zoho CRM while remaining laser-focused on merchant onboarding. Philosophy is unchanged: **Less Code, More Productivity — Less Files, More Productivity.** Every workflow should be reachable in a single click.

Key additions in this version:

- Three-tier org hierarchy: **Admin** creates and assigns **Managers**; each Manager has a **Team** of BDMs/Employees.
- Manager Dashboard updates **live in real time** as their team works — no page refresh.
- Admin sees every Manager on one screen and can open any Manager's full dashboard in **one click** (drill-down, not a separate login).
- Admin dashboard has smart auto-filters — **Project-wise**, **Reporting-Manager-wise**, Team-wise, Date-range — the view reshapes instantly.
- **Bulk Lead Upload**: pick a project → system auto-generates a ready-to-fill dummy Excel template matching that project's exact fields → upload back to bulk-create leads.
- **Custom Form Builder** (Google-Forms style): Admin builds a project-specific data-collection form by dragging in fields — Text, Number, Dropdown, File Upload, Multi-select, Date — with zero code. The form is scoped to one Project only.
- That same form appears live on the **BDM/Employee dashboard** for the matching project, so they can fill it directly against a lead.
- Team management: Create Team, Update Team, reassign members, and pull Team Reporting — all live.
- New performance layer: PostgreSQL, Redis, Django Channels (WebSockets), Celery, and optional Docker/Kubernetes for scale.

2. Organization Hierarchy & Roles

The system models a real company structure. Every user belongs to exactly one level, and data visibility cascades downward automatically.

Role	Scope	Key Permissions
Admin	Top of the company. Not tied to one project.	Create/edit Managers; assign Managers to Projects; view ALL Managers & drill into any Manager's dashboard in one click; global filterable dashboard (project / manager / date); build custom forms per project; manage bulk-upload templates; full Project CRUD; user & role management.
Manager	Owns one or more Projects and one or more Teams.	Create/Update Team, add/remove BDMs; real-time live dashboard of team's leads & conversions; verify uploaded documents; view team reporting/leaderboard; cannot see other Managers' teams unless Admin grants cross-project access.
TL (Team Lead)	Optional mid-layer inside a Manager's team.	Same as BDM plus: view teammates' leads, verify documents, follow-up escalation.
BDM / Employee	Works leads for the project(s) they're assigned to.	Own leads only; fill the project's custom form; schedule follow-ups; upload documents; sees live status updates on their own dashboard.

Cascading visibility rule: Admin → sees everything. Manager → sees only their own Team's leads/projects. TL → sees own + assigned BDMs. BDM → sees only leads assigned to them. This is enforced at the query level (DRF queryset filtering by request.user.role + reports_to chain), not just hidden in the UI.

3. Real-Time Live Dashboard Architecture

Every dashboard (BDM, Manager, Admin) updates instantly when a lead is created, a status changes, a document is verified, or a follow-up is logged — without the user refreshing the page.

How it works:

- **Django Channels** runs the backend on ASGI (Daphne/Uvicorn) instead of plain WSGI, adding WebSocket support alongside the existing REST API.
- **Redis** acts as the Channel Layer — the message bus that broadcasts an event (e.g. "lead_updated") to every connected dashboard that should see it.
- A lightweight Django signal (post_save on Lead / LeadDocument / Team) pushes a small event onto the channel layer; the relevant group (project_<id>, manager_<id>, admin) receives it and the frontend patches just that row/card — no full reload.
- Frontend keeps its normal React Query fetch for the first load, then subscribes to a WebSocket for live deltas — best of both worlds (fast paint + true real-time).
- **Celery + Redis broker** handles anything heavier than a quick signal: bulk Excel processing, sending the generated dummy template, recalculating leaderboard stats, and future email/SMS notifications — so the request/response cycle never blocks.

This mirrors the standard production pattern for Django live dashboards: ASGI + Channels + Redis channel layer for push updates, with Celery reserved for longer background jobs rather than the live push itself.

4. Updated Tech Stack

Layer	Technology
Backend Core	Python 3.14, Django 5.2, Django REST Framework, SimpleJWT
Real-time	Django Channels (ASGI via Daphne/Uvicorn), channels-redis
Background Jobs	Celery 5.x + Redis as broker/result backend
Database	PostgreSQL 16 (JSONB for dynamic forms), pgbouncer for connection pooling
Cache / Channel Layer	Redis 7
File Storage	Local disk for dev; S3-compatible storage (django-storages) for production documents/uploads
Bulk Excel	openpyxl / pandas — template generation + bulk import parsing
Containerization	Docker + docker-compose (dev); Kubernetes manifests (prod, only when scale needs it)
Frontend Core	Next.js 14, React 18, TypeScript, Tailwind CSS 3.4
Frontend Data/UI	React Query (server cache), Recharts (charts), Lucide React (icons), clsx + tailwind-merge
Animation	Framer Motion (page/card transitions, live-update pulse, drag-and-drop form builder)
Realtime client	native WebSocket hook (or socket.io-client alternative) wired to React Query cache updates

Why this stack: PostgreSQL's JSONB is what makes the no-code Custom Form Builder possible without a new table per project. Redis does double duty as the Channels layer AND the Celery broker/cache, so we don't introduce a second moving part. Docker/Kubernetes/Celery/Redis are added progressively — start with Docker Compose locally, and only reach for Kubernetes once real user load requires horizontal scaling.

5. Updated Database Models

Model	Key Fields
Project	name, slug, description, color, is_active, created_by(Admin)
User (custom)	role [ADMIN/MANAGER/TL/BDM], reports_to (self FK), mobile_number, is_active
Team	name, project FK, manager FK(User), members M2M(User), created_at
Merchant	project FK, name, mobile, email, brand_name, city
Lead	project FK, merchant FK, bdm FK, status, follow_up_date, notes, custom_data (JSONField — answers to that project's custom form)
LeadDocument	lead FK, gst/pan/cheque files, verification_status, verified_by FK
CustomForm	project FK (one active form per project), title, schema (JSONField — ordered field list), created_by(Admin), is_active
CustomFormField (embedded in schema JSON)	field_id, label, type [text/number/date/dropdown/multiselect/file/textarea], required, options[]
FormSubmission	custom_form FK, lead FK, submitted_by FK, answers (JSONField), submitted_at
BulkUploadJob	project FK, uploaded_by FK, file, status [pending/processing/done/failed], total_rows, success_rows, error_log (JSON), created_at — processed by Celery
Notification (future-ready)	user FK, message, is_read, link, created_at — pushed via Channels

Design note: Lead.custom_data and FormSubmission.answers use PostgreSQL JSONB so the Custom Form Builder can add/remove/reorder fields per project without any migration — the standard pattern for no-code form builders on top of Django.

6. Core New Workflows

6.1 Admin: One-Click Manager Drill-Down

- /admin shows a grid/list of every Manager with a live mini-summary (active leads, today's follow-ups, conversion %).
- Clicking a Manager card opens that Manager's exact dashboard view in place (same component the Manager sees), with a "Back to all Managers" breadcrumb — no separate login or URL needed.
- Global filter bar at the top of /admin: Project ■, Reporting Manager ■, Team ■, Date Range ■ — changing any filter re-renders every chart and table on the page instantly (client-side query params, server does the aggregation).

6.2 Bulk Lead Upload with Auto-Generated Template

- On /leads → Bulk Upload, user first selects a Project.
- Backend endpoint GET /api/projects/{id}/bulk-template/ reads that project's **CustomForm** schema (if any) plus the standard Lead/Merchant fields, and streams back a ready-made **.xlsx** with correct column headers, dropdown data-validation for choice fields, and one example row.
- User fills it and re-uploads via POST /api/projects/{id}/bulk-upload/. A **Celery task** validates row-by-row, creates Merchant+Lead(+FormSubmission) records, and writes a per-row error log for anything invalid.
- Progress is pushed live over WebSocket to a small progress bar ("182 / 500 rows processed") — the same pattern used for the dashboard live updates.

6.3 Custom Form Builder (per Project, Google-Forms style)

- Admin goes to /admin/forms, picks a Project (e.g. "Jio"), and drags fields onto a canvas: Short Text, Paragraph, Number, Dropdown, Multi-select, Date, File Upload.
- Each field just needs a label + type + required toggle + options (for dropdown/multi-select) — no code, no migration.
- Save writes the whole structure as one JSON schema on that Project's CustomForm row.
- The form is strictly scoped: it only ever appears for leads under that same Project, everywhere in the app (BDM dashboard, Manager view, bulk-upload template, lead detail sheet).
- The instant Admin publishes or edits the form, BDMs on that project see the updated form on their dashboard live (pushed over the same WebSocket channel) — they don't need to refresh.

6.4 Team Management

- Manager (or Admin) can Create Team → name it, attach it to a Project, add BDMs.
- Update Team → rename, swap members, reassign to a different Manager.
- Team Reporting page: leads worked, conversion rate, avg. response time, per-member leaderboard — live, currently backed by the real Lead data (dummy/mock only for pieces where no real API exists yet, e.g. future call-log integrations).

7. Updated API Surface

Method	Endpoint	Purpose
POST	/api/auth/login/	JWT login
GET	/api/me/	Current user + role + reports_to chain
GET	/api/dashboard/?project=	BDM/TL dashboard stats
GET	/api/manager/dashboard/?team=	Manager's live team dashboard
GET	/api/admin/dashboard/?project=&manager;=&team;=&from;=&to;=	Admin global dashboard, fully filterable

Method	Endpoint	Purpose
GET	/api/admin/managers/	List all Managers with live mini-summary
GET	/api/admin/managers/{id}/dashboard/	One-click drill into a specific Manager's dashboard
CRUD	/api/teams/	Create / Update / Delete Team
GET	/api/teams/{id}/reporting/	Team performance & leaderboard
CRUD	/api/projects/	Project management (Admin only)
CRUD	/api/leads/?project=	Leads with project filter
PATCH	/api/leads/{id}/follow_up/	Schedule follow-up
POST	/api/leads/{id}/upload_documents/	Upload GST/PAN/cheque docs
PATCH	/api/leads/{id}/verify_documents/	TL/Manager verify
GET/PUT	/api/projects/{id}/custom-form/	Get or update that project's dynamic form schema
POST	/api/leads/{id}/form-submission/	Submit answers to the project's custom form
GET	/api/projects/{id}/bulk-template/	Auto-generate & download dummy Excel template
POST	/api/projects/{id}/bulk-upload/	Upload filled Excel → Celery bulk-creates leads
GET	/api/bulk-jobs/{id}/	Poll/subscribe to bulk-upload job status
WS	/ws/dashboard/{scope}/	WebSocket: live push for admin / manager / bdm dashboards

8. Frontend Pages (Minimal File Structure Kept)

Route	Purpose
/	Login (JWT)
/dashboard	BDM/TL — metrics cards, pie/bar charts, live-updating badge, project's custom form inline on each lead
/leads	Leads table — Add, Edit, Follow-up, Delete, View sheet, Bulk Upload tab
/team	Manager — live team dashboard, Create/Update Team, Team Reporting/leaderboard
/admin	Admin — global filterable dashboard + Manager grid with one-click drill-down
/admin/projects	Manage Projects (Manager-role gate: Admin only)
/admin/forms	Custom Form Builder — pick project, drag-drop fields, publish

Frontend file additions (still one-file-per-concern):

- components/team.tsx — Team CRUD + reporting UI
- components/form-builder.tsx — drag-drop custom form canvas (Admin)
- components/dynamic-form.tsx — renders any project's CustomForm schema (used on BDM dashboard + lead sheet + bulk template preview)
- components/bulk-upload.tsx — template download + upload + live progress bar
- lib/ws.ts — single WebSocket hook (useLiveDashboard(scope)) reused by BDM/Manager/Admin dashboards

9. Design System — Clean, Professional, Animated

- Background bg-slate-50, cards white/rounded-2xl/soft shadow, generous whitespace — no clutter.
- Primary blue-600, Success emerald, Warning amber, Danger rose — same palette, used consistently for status chips.
- Font: Inter via next/font/google.
- Micro-animations with Framer Motion: cards fade/slide in on load, a soft pulse highlight on any row that just updated live, smooth drag feedback in the Form Builder, animated number count-up on dashboard KPIs, skeleton loaders instead of blank screens.
- Every primary action reachable in one click from where the user already is (no buried menus) — Bulk Upload, Add Lead, Create Team, and Build Form are all one click from their respective pages.
- Fully responsive: sidebar collapses to a bottom bar / hamburger on mobile.

10. Performance & Scaling Plan

- **Now:** PostgreSQL (indexed FKs + JSONB GIN index on custom_data), Redis cache for dashboard aggregates, Django Channels for live push, Celery for bulk upload & heavy report generation, Docker Compose for one-command local/staging spin-up.
- **When load grows:** pgbouncer for connection pooling, read replicas for heavy dashboard queries, Kubernetes (HPA) to auto-scale the ASGI workers and Celery workers independently, S3 for document storage instead of local disk, CDN for the Next.js static assets.
- **Query discipline:** every list endpoint paginated + select_related/prefetch_related by default; dashboard aggregates computed with DB-level annotate(), not Python loops.

11. Development Phases (Updated)

Phase	Scope
Phase 1	Django + Next.js scaffold, models, login, dashboard, leads table (existing, done)
Phase 2	Multi-project support, project switcher, admin panel (existing, done)
Phase 3	Full Leads CRM — Add/Edit/Follow-up/Delete, overdue highlight (existing, done)
Phase 4	Admin Dashboard — all-projects overview, charts, leaderboard (existing, done)
Phase 5	UI/CSS polish pass (existing, done)
Phase 6	PostgreSQL migration + Team model + Admin↔Manager one-click drill-down + global admin filters
Phase 7	Django Channels + Redis channel layer — live dashboards for BDM/Manager/Admin
Phase 8	Custom Form Builder (CustomForm + dynamic-form.tsx) scoped per project
Phase 9	Bulk Upload — template auto-generation, Celery import job, live progress
Phase 10	Docker Compose everywhere; Kubernetes manifests + horizontal scaling only if/when needed

12. How to Run (Updated)

Backend

```
cd CRM_Backend
docker-compose up -d  # starts Postgres + Redis
pip install -r requirements.txt
python manage.py migrate && python manage.py seed
daphne CRM_Backend.asgi:application --port 8000  # ASGI server, replaces runserver in prod
celery -A CRM_Backend worker -l info  # background jobs (bulk upload, reports)
```

Frontend

```
cd CRM_Frontend
npm install && npm run dev  → http://localhost:3000
Env: NEXT_PUBLIC_API_URL=http://127.0.0.1:8000
Env: NEXT_PUBLIC_WS_URL=ws://127.0.0.1:8000
```

Demo credentials (password: password123) — manager / tl / bdm. Demo Projects: Amazon, Flipkart, Meesho (5 leads each, one dummy CustomForm each for testing).

13. Known Issues & Fixes (Carried Over + New)

- Cannot find module 217.js → delete .next folder, restart dev server.
- CSS 404 / unstyled page → clear .next, fix globals.css, add autoprefixer.
- django-admin not in PATH → use python -m django.
- PowerShell && not supported → use semicolon separator.
- Migration stuck → delete old migration, fresh 0001_initial.
- WebSocket 403 on connect → check Channels AuthMiddlewareStack is wrapping the JWT token correctly; pass the token as a query param since browsers can't set custom WS headers.
- Celery task silently not running → confirm Redis broker URL matches docker-compose service name, and that celery worker is actually started alongside daphne.
- JSONB field not filterable as expected → add a GIN index via a migration: class Meta: indexes = [GinIndex(fields=["custom_data"])].

14. Not Yet Built (Future Scope)

- Kanban board view for leads
- Email/SMS notifications (structure ready via Notification model + Channels, wiring pending)
- Kubernetes autoscaling manifests (only needed once traffic requires it)
- Mobile app / PWA wrapper
- Per-field conditional logic in Form Builder ("show field B only if field A = X") — v2 of the builder
- Audit log / activity timeline per lead

*This document supersedes the original single-tier project summary and should be treated as the source of truth for Cursor AI development going forward. See the companion file **Cursor_AI_Build_Command.md** for the exact prompt to paste into Cursor.*